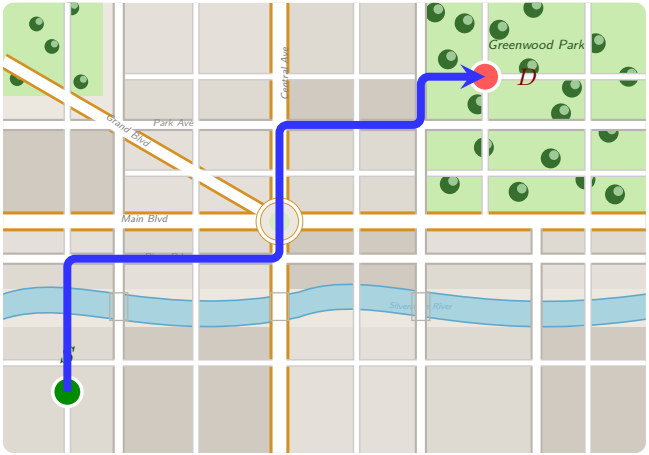
Algorithms

Dr. Mudassir Shabbir

LUMS University

March 25, 2026

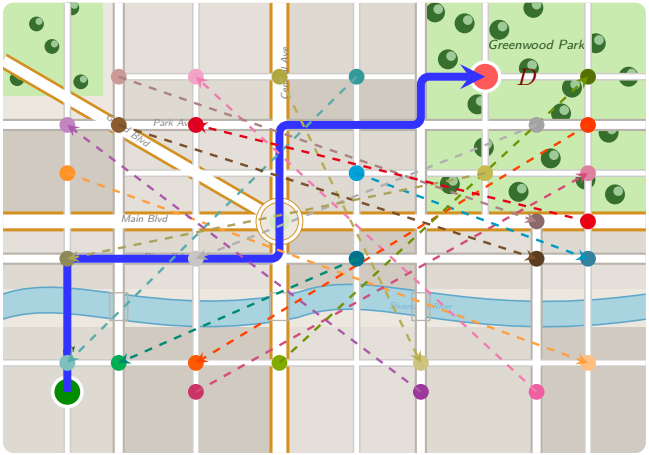
Motivation: Routing at Scale



Navigation apps (Google Maps, Waze, ...) must answer:

"What is the shortest path from A to B?"

Motivation: Routing at Scale

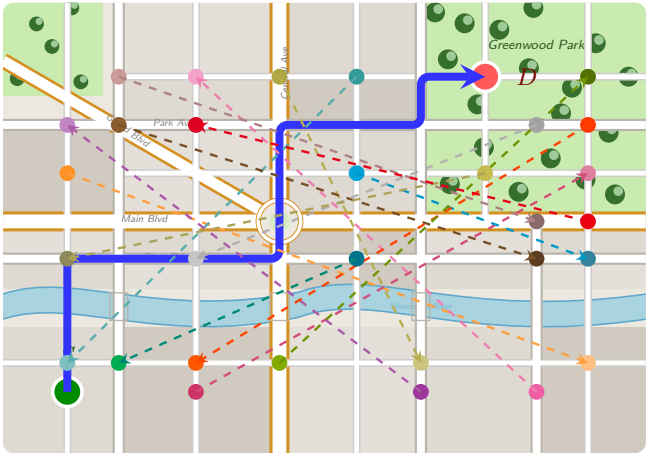


Navigation apps (Google Maps, Waze, ...) must answer:

"What is the shortest path from A to B?"

Millions of users, simultaneously.

Motivation: Routing at Scale



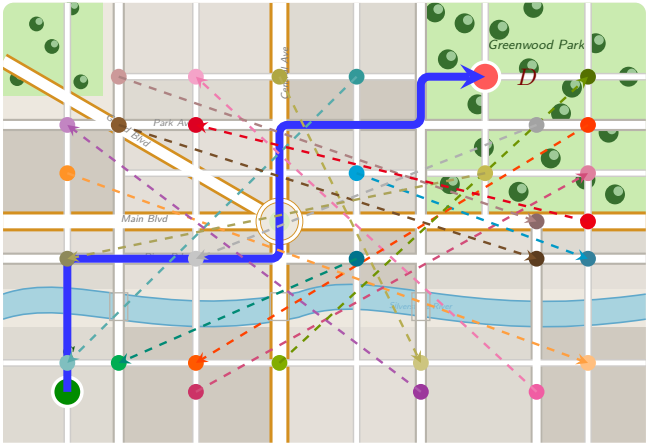
Navigation apps (Google Maps, Waze, ...) must answer:

"What is the shortest path from A to B?"

Millions of users, simultaneously.

Re-running a shortest-path algorithm per query is too slow at scale.

Motivation: Routing at Scale



Idea: Precompute all shortest paths (APSP).

Navigation apps (Google Maps, Waze, ...) must answer:

"What is the shortest path from A to B?"

Millions of users, simultaneously.

Re-running a shortest-path algorithm per query is too slow at scale.

All-Pairs Shortest Paths

Given a weighted graph $G = (V, E, w)$, compute shortest-path distances between **every ordered pair** (u, v).

APSP Problem Statement

Input: A directed (or undirected) weighted graph $G = (V, E, w)$.

APSP Problem Statement

Input: A directed (or undirected) weighted graph $G = (V, E, w)$.

Output: A distance matrix D where

$$D[u][v] = \text{length of a shortest path from } u \text{ to } v.$$

APSP Problem Statement

Input: A directed (or undirected) weighted graph $G = (V, E, w)$.

Output: A distance matrix D where

$$D[u][v] = \text{length of a shortest path from } u \text{ to } v.$$

Corner cases:

- If v is unreachable from u, then $D[u][v] = \infty$.
- If a negative cycle is reachable from u and can reach v, then shortest path is undefined (effectively $-\infty$).

APSP Problem Statement

Input: A directed (or undirected) weighted graph $G = (V, E, w)$.

Output: A distance matrix D where

$$D[u][v] = \text{length of a shortest path from } u \text{ to } v.$$

Corner cases:

- If v is unreachable from u, then $D[u][v] = \infty$.
- If a negative cycle is reachable from u and can reach v, then shortest path is undefined (effectively $-\infty$).

Use cases

Distance queries in road networks, routing between all servers, and optimization pipelines.

Floyd–Warshall: Dynamic Programming Idea

Index vertices as $1, 2, \ldots, n$.

Floyd–Warshall: Dynamic Programming Idea

Index vertices as $1, 2, \ldots, n$.

Define $D^{(k)}[i][j]$ as the shortest path from i to j using only intermediate vertices from $\{1, \ldots, k\}$.

Floyd–Warshall: Dynamic Programming Idea

Index vertices as $1, 2, \ldots, n$.

Define $D^{(k)}[i][j]$ as the shortest path from i to j using only intermediate vertices from $\{1, \ldots, k\}$.

Recurrence:

$$D^{(k)}[i][j] = \min\Big(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\Big).$$

Floyd–Warshall: Dynamic Programming Idea

Index vertices as $1, 2, \ldots, n$.

Define $D^{(k)}[i][j]$ as the shortest path from i to j using only intermediate vertices from $\{1, \ldots, k\}$.

Recurrence:

$$D^{(k)}[i][j] = \min\Big(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\Big).$$

Interpretation:

- Either best path avoids vertex k.
- Or best path goes through k (split into $i \to k$ and $k \to j$).

Floyd–Warshall: Dynamic Programming Idea

Index vertices as $1, 2, \ldots, n$.

Define $D^{(k)}[i][j]$ as the shortest path from i to j using only intermediate vertices from $\{1, \ldots, k\}$.

Recurrence:

$$D^{(k)}[i][j] = \min\Big(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\Big).$$

Interpretation:

- Either best path avoids vertex k.
- Or best path goes through k (split into $i \to k$ and $k \to j$).

After $k = n$, matrix $D^{(n)}$ is the APSP answer.

Floyd–Warshall Pseudocode

```
procedure FloydWarshall(W)
    D ← W    (adjacency matrix, with ∞ for missing edges)
    for i = 1 to n do: D[i][i] ← 0
    for k = 1 to n do
      for i = 1 to n do
        for j = 1 to n do
          if D[i][k] + D[k][j] < D[i][j] then
            D[i][j] ← D[i][k] + D[k][j]
    return D
```

Floyd–Warshall Pseudocode

```
procedure FloydWarshall(W)
    D ← W    (adjacency matrix, with ∞ for missing edges)
    for i = 1 to n do: D[i][i] ← 0
    for k = 1 to n do
      for i = 1 to n do
        for j = 1 to n do
          if D[i][k] + D[k][j] < D[i][j] then
            D[i][j] ← D[i][k] + D[k][j]
    return D
```

Complexity: triple loop $\Rightarrow O(V^3)$ time, $O(V^2)$ space.

Negative Cycle Detection in APSP

For Floyd–Warshall, after completion inspect diagonal entries.

Negative Cycle Detection in APSP

For Floyd–Warshall, after completion inspect diagonal entries.

$$D[v][v] < 0 \quad \Longleftrightarrow \quad v \text{ lies on (or reaches and returns through) a negative cycle.}$$

Negative Cycle Detection in APSP

For Floyd–Warshall, after completion inspect diagonal entries.

$$D[v][v] < 0 \quad \Longleftrightarrow \quad v \text{ lies on (or reaches and returns through) a negative cycle.}$$

Why diagonal?

- $D[v][v]$ is shortest closed walk cost from v to itself.
- A negative closed walk implies unboundedly decreasing paths.

Negative Cycle Detection in APSP

For Floyd–Warshall, after completion inspect diagonal entries.

$$D[v][v] < 0 \quad \Longleftrightarrow \quad v \text{ lies on (or reaches and returns through) a negative cycle.}$$

Why diagonal?

- $D[v][v]$ is shortest closed walk cost from v to itself.
- A negative closed walk implies unboundedly decreasing paths.

Practical output convention

Report pair (i, j) as $-\infty$ if there exists a vertex k with $D[k][k] < 0$, reachable from i and reaching j.